Available online at www.sciencedirect.com
ScienceDirect
Information Processing Letters 103 (2007) 142–149
Information Processing Letters
www.elsevier.com/locate/ipl

Improved approximate common interval

Amihood Amir [a,b,*,1], Leszek Gasieniec [c,2], Riva Shalom [a,3]

[a] *Department of Computer Science, Bar-Ilan University, Ramat-Gan 52900, Israel*
[b] *College of Computing, Georgia Tech, Atlanta, GA 30332-0280, USA*
[c] *Department of Computer Science, The University of Liverpool, L69 7ZF, Liverpool, UK*

Received 17 January 2006; received in revised form 1 August 2006; accepted 5 March 2007

Available online 20 March 2007

Communicated by S.E. Hambrusch

Keywords: Design of algorithms; Pattern matching; Computational biology; Hamming distance; Gene evolution

1. Introduction and related work

During the course of evolution, speciation results in the divergence of genomes that initially have the same gene order and content. If there is no selective pressure, successive rearrangements that are common in prokaryotic genomes will eventually lead to a randomized gene order. Therefore the presence of a region of conserved gene order is a source of evidence of some non-random signal that allows, for example, the prediction of groups of functionally associated genes [8].

Usually two closely related prokaryotes share many gene clusters, which are sets of genes in close proximity to each other, but not necessarily contiguous nor in the same order in both genomes [6]. The existence of such gene clusters has been explained in various ways [7] which show that the conservation of gene order is a source of information for many fields in genomic research. The criteria for a set of genes to form a gene cluster varies, for example, Beal et al. [2,4], demand that the distance of adjacent genes in the set is no more than a certain threshold. Genomes are usually represented by permutations [7,2], however Schmidt and Stoye [7] suggested considering them as strings, with possible repeats of symbols representing paralogous genes which, they claim, is closer to reality. They attack the problem of detecting gene clusters in a new fashion while modeling gene intervals by the set of characters (genes) compiling them. They defined the problem as finding common character sets between two strings.

Definition 1. (See [7].) Given a string S the *character set of a substring* $S[i, j]$ is $CS(S[i, j]) = \{S[k] \mid i \leqslant k \leqslant j\} \subset \Sigma$.

Throughout the paper a substring $S[i, j]$ is also referred to as interval $[i, j]$. A character set represents the set of all genes occurring in a given genome interval, where the order and the number of occurrences of identical copies of a gene are irrelevant.

Schmidt and Stoye give an $O(n^2)$ time algorithm that locates all intervals of two strings that share the same

* Corresponding author at: Department of Computer Science, Bar-Ilan University, Ramat-Gan 52900, Israel. Tel.: +972 3 531 8770.
E-mail addresses: amir@cs.biu.ac.il (A. Amir), leszek@csc.liv.ac.uk (L. Gasieniec), gonenr1@cs.biu.ac.il (R. Shalom).

[1] Partly supported by NSF grant CCR-01-04494 and ISF grant 35/05.

[2] Tel.: +44 151 794 3686.

[3] Tel.: +972 3 531 8408.

0020-0190/$ – see front matter
doi:10.1016/j.ipl.2007.03.006

$$
\begin{array}{lcl}
\{1\} & \longleftrightarrow & \{1,2\},\{1,3\} \\
\{2\} & \longleftrightarrow & \{1,2\},\{2,3\} \\
\{3\} & \longleftrightarrow & \{1,3\},\{2,3\} \\
\{1,2\} & \longleftrightarrow & \{1\},\{2\},\{1,2,3\} \\
\{1,3\} & \longleftrightarrow & \{1\},\{3\},\{1,2,3\} \\
\{2,3\} & \longleftrightarrow & \{2\},\{3\},\{1,2,3\} \\
\{1,2,3\} & \longleftrightarrow & \{1,2\},\{2,3\},\{1,3\}
\end{array}
$$

Fig. 1. Output for $K = 1$, $S = \{S_1 = 121312, S_2 = 232321\}$.

character set, where n is the length of the strings. Their algorithm can be extended to solving the common intervals of L strings in $O(Ln^2)$ time and space. In [7] the question of generalizing their model to report also gene clusters with a small symmetric set difference, was raised. It may be assumed that if some differences between the character sets would be allowed, longer intervals will be considered as common. This leads to the problem of *Approximate Common Intervals*.

Definition 2. The *Approximate Common Intervals Problem* (ACI) is defined as follows:

Input: A set of strings $S = \{S_1, \ldots, S_L\}$ of size n each over alphabet Σ, and a constant K.

Output: For every distinct character set cs appearing in S, report all character sets cs', occurring in S, such that $cs \Delta cs' \leqslant K$, where Δ stands for symm. difference.

For example, let $S = \{S_1 = 121312, S_2 = 232321\}$. We say S_1 contains character sets $\{1\}$ at $[1,1], [3,3], [5,5]$; $\{2\}$ at $[2,2], [6,6]$; $\{3\}$ at $[4,4]$; $\{1,2\}$ at $[1,3], [5,6]$; $\{1,3\}$ at $[3,5]$ and $\{1,2,3\}$ at $[1,6]$. S_2 contains character sets $\{1\}$ at $[6,6]$; $\{2\}$ at $[1,1], [3,3], [5,5]$; $\{3\}$ at $[2,2], [4,4]$; $\{2,3\}$ at $[1,5]$; $\{1,2\}$ at $[5,6]$ and $\{1,2,3\}$ at $[1,6]$. For $K = 1$ the output would be as in Fig. 1. For simplicity we do not denote the source intervals of the character sets here.

The output may seem to suffer from redundancy, however any other way of clustering the results will lose information.

We suggest considering the ACI problem as an approximate dictionary matching problem for $K = 1$ where we get an algorithm for $L = 2$ whose time and space is $O(n^3)$, where n is the length of the strings. This algorithm can be extended to deal with $L > 2$ consuming time and space of $O(Ln^3)$.

For cases of $1 < K \leqslant \log n$ one may consider the ACI problem as an approximate indexing problem. This can lead to an algorithm whose time is $O(Ln^3 \log n + Ln^2(6^K \log^K n)/K!) + occ)$, and whose space complexity is $O(Ln^3 + (c^K Ln^3 \log^K n)/K!)$ where n is the length of the longest sequence and c is a constant.

Nevertheless, in both problems every character set of a certain string was compared to all other character sets of the other string. Since the distance allowed in the problem is symmetric difference, it turns out that not all comparisons are necessary. In this paper we exploit this fact and show that the Approximate Common Interval Problem for L strings can be solved in $O(Ln^3 + occ)$ time and $O(Ln^3)$ space, where n is the length of every string and occ is the size of the output.

The paper is organized as follows: In the next section we give some preliminaries including a procedure for extracting all maximal character sets of the input strings. In Section 3 we deal with the ACI problem for a single input string. Section 4 extends the solution for multiple input strings. In Section 5 we give a brief description of other questions easily supported by our construction. Section 6 concludes the paper.

2. Preliminaries

Naturally, before solving the ACI problem all character sets contained in the input strings should be extracted. We save the characters sets found, in a bit vector of size Σ, called *fingerprint*, where the ith entry is set iff the ith symbol appears in the character set. The ACI problem now transforms into the problem of finding intervals, of different source strings, whose bit vectors match with at most K substitution mistakes (Hamming distance).

We assume, without loss of generality, that $\Sigma = \{1, 2, \ldots, |\Sigma|\}$ since an $O(n \log n)$ preprocessing of any string is sufficient to construct an equivalent string over alphabet $\Sigma = \{1, 2, \ldots, |\Sigma|\}$. This preprocessing time will be subsumed by the overall time of our algorithm.

Note that in our application, the number of different genes, the alphabet size, is closely related to the length of the genome. Thus we assume that $|\Sigma|$ is $\Theta(n)$.

Another important observation is that we consider only maximal intervals consisting of a certain character set, meaning we do not include two substrings s', s'' having the same character set, when one is a substring of the other. For example, let $S = 42323632$, $CS(S[1,3]) = \{2,3,4\} = CS(S[1,5])$, here we select only $S[1,5]$ to represent the character set $\{2,3,4\}$. Every input string is processed separately.

Observation 1. *The number of distinct character sets of size f is bounded by $n - f + 1$, where n is the length of the string.*

Proof. A character set of size f, denoted by cs_f, can be obtained from an interval containing at least f symbols. The number of distinct f long intervals contained in a string of n characters is $n - f + 1$. A character set of size f can appear also in longer intervals, but in such cases the number of intervals containing cs_f will decrease. Moreover, there may be some intervals containing the same character set, thus $n - f + 1$ serves as a bound, which is tight only for permutations. □

Listing all character sets of size 1 is simply converting the string into its run length encoding. By a single pass over the string all adjacent characters that are identical are considered as one character, and the indices are now intervals of the original string. Extracting the character sets of sizes two and more can be done as in [1] as follows: We define a counters array *counters* of size Σ initialized to 0. Every entry *counters*$[x]$ will represent the number of occurrences of symbol x in the current substring, implying that the number of non-zero entries in *counters* is the size of the current character set. We retrieve all character sets of size $f, 1 < f < |\Sigma|$, in a single pass over S and save them in a list named $List^f$. For each f, we define a window of variable size including exactly f distinct characters, each of which may occur more than once. Let window (i, j) define an f sized character set. We move the right boundary of the window to the right, increasing j, as long as no new symbol is encountered, and the new window still defines the same f sized character set, though *counters* is updated. Once a new symbol x is encountered (*counters*$[x]$ equaled 0 so far), the right boundary of the previous window $(j - 1)$ is set to be the end of the maximal substring bearing that character set. The current window (i, j) now represents a $f + 1$ sized character-set, so we move the left boundary to the right, increasing i, every symbol that is passed over decreases its counter by one. This is done until one symbol is dropped, one of *counters* entry was reduced to 0, and we have a new window which defines an f sized character set, with its right boundary set as before. Whenever a character set of size f is declared, we associate to its fingerprint, extracted from *counters*, the interval whose endpoints are the window boundaries and add it to the fth list named $List^f$.

This procedure is repeated for every f, thus its time complexity is $O(n|\Sigma|)$. In the worst case this is asymptotic in the output size as there may be $O(n^2)$ distinct character sets. For example, consider the string $123 \ldots n$.

3. ACI for a single string

We suggested considering the ACI problem as an approximate dictionary matching problem or as an approximate indexing problem. However, in both problems every character set of a certain string needs to be compared to all other character sets of the other string. Since the distance allowed in the problem is symmetric difference, the following important observation can spare us the need of performing many unnecessary comparisons.

3.1. $K = 1$

Observation 2. *Demanding exactly a single symmetric difference* ($K = 1$)*, a character set of size* f *should be checked only against character sets of size* $f + 1$ *and those of size* $f - 1$.

Proof. The symmetric difference between a character set cs_1 of size f and another cs_2 of size g, where $g > f + 1$ or $g < f - 1$ is bound to be greater than 2, as the number of symbols compiling these character sets differers by at least two, regardless of their actual symbols. If cs_2 is of size f as well, it may have exactly the same characters as in cs_1, where the symmetric difference is zero. If the characters are not identical, there must be at least two such that one appears in cs_1 and not in cs_2 and the other appears in cs_2 and not in cs_1, as we have $|cs_1| = |cs_2|$, yielding a symmetric difference of at least two between cs_1 and cs_2. It follows that the only possibilities to obtain a single symmetric difference between two character sets is when their sizes differ by one and the smaller is contained in the larger. □

We would like to represent the relations between the character sets found, in a way it would be easy to ask queries of a single symmetric difference.

To this aim we construct graph $G = (V, E)$ representing all single symmetric differences between character sets of the string S. The nodes correspond to the distinct characters sets obtained in the previous section, having at most $O(n^2)$ such. An edge between two nodes will indicate that these character sets have a single symmetric difference between them.

3.2. Constructing G

According to the above observation we construct G in 'layers' where the ith layer stands for the i-*cs*es of S. First we construct V by creating for every character set of the $List^f$s, $1 \leqslant f \leqslant |\Sigma|$, a node. The node contains

the fingerprint of the character set. We also save at the node the interval representing it. In case more than one interval has the same character set cs, there will be a single node in V referring to this character set, and some intervals will be attached to the node. We can obtain repeating character sets by sorting $List^f$ before creating the nodes or by using the algorithm of Schmidt and Stoye [7], applied to S with itself, taking $O(n^2)$ time. There are other methods for extracting all distinct character sets of a given string such as the algorithm suggested by Kolpakov and Raffinot [5], however, they do not improve the time nor space complexity for our case, therefore, we chose to use the uncomplicated approach of using the window procedure and sorting. To conclude, as there are $O(n^2)$ possible consecutive intervals of varying lengths, and thus $O(n^2)$ distinct character sets, we get $|V| = O(n^2)$.

Regarding E, there exists an edge (v^i, v^j) iff the symmetric difference between the character set of v^i and that of v^j is one. From Observation 2 it follows that $(v^i, v^j) \in E$ iff $v^i \in list^f$ and $v^j \in list^g$ where $g \in \{f+1, f-1\}$. Every edge also bears the symbol added/removed to/from the cs_f, transforming it into a cs_g. We form the edges between layer f and layer $f+1$, for every $1 \leqslant f < |\Sigma|$ in the following manner:

Since the character sets are saved in bit vectors of size $|\Sigma|$, detecting, for every cs_f all $cs_(f+1)$s having a single symmetric difference from it, transforms, here again, into the problem of searching among all the $cs_(f+1)$s those with a hamming distance one from the current cs_f. We consider this problem as an approximate dictionary matching problem, where the $cs_(f+1)$s form the dictionary, and all cs_fs are considered as query text, each at its time. As all dictionary patterns and texts are binary and of the same length, they are all fingerprints bit vectors, we can use the Brodal Gasieniec [BG] algorithm [3]. Their algorithm construct two tries, one according to the dictionary patterns, and the other according to their reverse. Roughly, they search the query q forward in the first try and its reverse in the second, trying to locate an index i where $q[1 \ldots i-1]$ appears in the first try and $q[|q| \ldots i+1]$ is found in the counterpart try. Note, that when locating such index i this is the index of the mismatch between q and the found pattern, thus this algorithm, cannot only detect the existence of edges between a cs_f and $cs_(f+1)$s, but also trace the labels attached to them. A simple modification to the algorithm would make it return that index associated to a matching pattern $(cs\_(f+1))$ as well.

The Brodal Gasieniec algorithm requires $O(D)$ time and space, where D is the size of the dictionary, and support a query q in time $O(|q| + occ)$ for occ the number of occurrences of q in the dictionary.

CONSTRUCT_GRAPH(S)

1 **For** $f = 1$ to $|\Sigma|$ **do**:
2 $List^f \leftarrow$ all cs_f obtained from S.
3 Sort $List^f$ alphabetically.
4 Unite identical cs_fs and copy their intervals to the saved item.
5 For every distinct $cs\_f \in List^f$ **do**:
6 Create a new node v_{cs} and associate to it its cs_f fingerprint and intervals.
7 $V \leftarrow v_{cs}$.
8 **For** $f = 1$ to $|\Sigma - 1|$ **do**:
9 Construct a dictionary D from all $cs\_(f+1) \in List^{f+1}$
10 Preprocess D according to BG alg.
11 **For** every $cs\_f \in List^f$
12 Apply a text query of cs_f on D.
13 **For** every $cs\_f + 1$ reported as an approx. match of cs_f:
14 $i \leftarrow$ the index reported associated to $cs\_(f+1)$.
15 $E \leftarrow (cs\_f, cs\_(f+1))$ labeled by $\Sigma[i]$.
16 **Return** G.

Fig. 2. Constructing G from S.

The construction of G is formally described in Fig. 2 (an example of G constructed upon two strings is depicted in Fig. 4).

Observation 3. *The number of edges between all nodes representing cs_fs (layer f) and nodes representing $cs_(f+1)$s (layer $f+1$) is bounded by* $O(nf)$.

Proof. According to Observation 1, the number of $cs_(f+1)$s is $n - f$. A $cs\_(f+1)$ is constructed from a cs_f to which a symbol is added. Therefore, every $cs\_(f+1)$ can have at most $f+1$ distinct edges emanating from cs_fs, yielding $(n-f)(f+1)$ possible edges between layer f and layer $f+1$. □

Lemma 1. *Graph G retrieved by the* CONSTRUCT_GRAPH(S) *procedure, preserves the single symmetric differences relation between all character sets occurring in string S.*

Proof. This lemma derives directly from the construction procedure. □

Lemma 2. *G is constructed in* $O(n^3)$ *time and space.*

Proof. The first step of extracting all character sets of S, grouped according to their sizes, is done in $O(n^2)$ using the window procedure. Sorting the lists as in line 3 takes $O(n^2 \log n)$ and then adding repeating character sets intervals to the first appearance of that cs is linearly done for every list. The maximal number of distinct character sets is $O(n^2)$, according to Observation 1.

Constructing V (lines 5–7) is linear in the number of the character sets found.

In order to create edges between layers f and $f+1$ we construct a dictionary and preprocess it as in [BG] algorithm (lines 9–10) in time $O(n^2)$, linear in the dictionary size. Each of the $n-f+1$ cs_fs serves as a query text, and performing a query requires $O(n+occ)$ time. Thus, the time required to construct edges between two consecutive layers is $O(n^2+edges)$ and according to Observation 3 it is bounded by $O(n^2+nf)=O(n^2)$. Note, that every edge is added exactly once.

All in all the procedure of constructing both V and E, including edges between all consecutive layers consumes $O(n^2\log n)+O(n^3)=O(n^3)$ time.

Regarding space: The $List^f$s consumes $O(n^2)$ space and this is also the number of intervals we save. V contains $O(n^2)$ vertices, each contains a character set bit vector which is $O(n)$ long.

While detecting the edges between layers f, $f+1$ a dictionary and tries consuming $O(n^2)$ space are constructed. This procedure is repeated $n-1$ times, yet before constructing the dictionary of the next layer, the previous one can be deleted, thus, merely $O(n^2)$ space is required for the preprocesses done.

Due to Observation 3, the number of edges between consecutive layers cannot exceed $O(n^2)$, therefore the total number of edges is bounded by $O(n^3)$. The label attached to every edge bears a single character, thus not requiring additional space. All in all we get that the space required for $|V|$ equals the demands of $|E|=O(n^3)$, so the space requirements for the construction of G is $O(n^3)$. □

As a consequence of Lemma 1, having constructed G according to S, if we want to retrieve all character sets appearing in S with a single symmetric difference between them, we can traverse G as an undirected graph, and for every node encountered report all its neighboring nodes. Such a traversal would cost $O(n^3)$ time and the output is also bounded by $O(n^3)$ as a certain character set can have at most $|\Sigma|-1$ character sets differing by a merely one symbol. As a consequence, such a procedure would require $O(n^3)$ time and space, which is linear in the output size. The next subsection extends this notion of finding character sets K symbols apart from each other.

3.3. $K > 1$

Lemma 3. *Given a node v representing cs_v, a node $u \in V$ located at the end of a path of length K, starting in v, where all labels on the path are distinct, stands for a character set cs_u where $cs_v \Delta cs_u = K$.*

Proof. The proof is inductive. G was built in a way that every path starting at v of length 1, an edge, ends at a node representing a character set with a single symmetric difference from v. Suppose u is $K-1$ distinct edges apart from v. By the induction hypothesis $cs_v \Delta sc_u = K-1$. The importance of the distinct labels on the path derives from the meaning of those labels. The label of an edge represents the character added/removed to/from one cs to another, to form a different character set. If on the path from v, symbol x appears on the edges twice, it means x was removed and then added or vice versa, so the symmetric distance between v and u is actually $K-2$ (and would be reached via another legitimate path $K-2$ long).

Continuing on the path through (u, w), labeled by a character not encountered already. w differs from u by a single symbol, yet it is a symbol not added/removed to/from the character sets on the way from v to u, therefore, w represents a character set cs_w that has K changes from cs_v thus, $cs_v \Delta sc_w = K$. □

According to Lemma 3, in order to retrieve all character sets within K symmetric difference from a certain character set cs, all we have to do is to start a modified Depth First Search from the node corresponding to cs till depth of K, pruning certain edges. Every node encountered should be reported, as it represents a character set with the proper symmetric difference from cs. Supervising the labels encountered on the current path traversed can be done by saving a bit vector of size $|\Sigma|$ named *labels*. When a label x is met, we set *labels*$[x]$. When we backtrack through x, *labels*$[x]$ is cleared. If *labels*$[x]$ is already set when we reach another label x, we backtrack from the last x edge (not changing *labels*$[x]$), by this prune the illegitimate path. As every node encountered is part of the output, this procedure is linear in the output size. The solution to the Approximate Common Intervals problem for a single string can be viewed in Fig. 3.

4. ACI for multiple strings

The Original Approximate Common Intervals problem relates to several input strings among which we seek intervals having approximate common character sets. We will show that our graph representation of the character sets can be extended to more than a single string, with no restrictions regarding the number of strings participating.

```
ACI(S, K)
 1  G ← Construct_Graph(S).
 2  Let labels be a bit vector of size |Σ|.
 3  For every v ∈ V
 4     Clear labels.
 5     Perform a DFS traversal starting from v till depth K.
 6        u ← the current active node.
 7        Output u and its intervals as approximately common to v's
          intervals.
 8        w ← the next node on the traversal.
 9        x ← the label of (u, w).
10        If labels[x] = 1 then:
11           Stop proceeding in this direction. Check the next edge
             of u.
12        else
13           labels[x] ← 1
14           Proceed to w and goto line 6.
```

Fig. 3. Approximate common intervals with a single string algorithm.

Given a set of strings $S = \{S_1, S_2, \ldots, S_L\}$ we would like to obtain a graph $G = (V, E)$, representing all character sets occurring at intervals from all the input strings, preserving the single symmetric difference relation between them. First, each of the strings undergoes the 'window' procedure, independently, yielding L sets of lists, $List_1^f$s, $List_2^f$s, …, $List_L^f$s for $1 \leqslant f \leqslant n$ correspondingly.

We form n new lists $List^f$, where $List^f$ gathers all $List_i^f$, for $1 \leqslant i \leqslant L$, where every item of a list includes the index of the string S_i from which it was extracted, in addition to the character set it represents and the interval of its occurrence. Each of the new lists contains all character sets of a certain size f, appearing in intervals from all the S strings, thus they constitute the nodes of layer f. The rest of the construction is exactly as it was for a single string, described in Fig. 2.

For example, suppose $S = \{S_1 = 1321234123145, S_2 = 42143124135\}$ we have $List^1 = \{1, 2, 3, 4, 5\}$, $List^2 = \{\{1, 2\}, \{1, 3\}, \{1, 4\}, \{2, 3\}, \{2, 4\}, \{3, 4\}, \{3, 5\}, \{4, 5\}\}$, $List^3 = \{\{1, 2, 3\}, \{1, 2, 4\}, \{1, 3, 4\}, \{1, 3, 5\}, \{1, 4, 5\}, \{2, 3, 4\}\}$, $List^4 = \{\{1, 2, 3, 4\}, \{1, 3, 4, 5\}\}$, $List^5 = \{1, 2, 3, 4, 5\}$. The graph constructed according these lists can be viewed in Fig. 4. Note that to every node v all occurrences of cs_v are attached, however they are missing in the figure, for visual c.

Lemma 4. *G^S induced by the* CONSTRUCT_GRAPH(S) *procedure for $S = \{S_1, \ldots, S_L\}$, preserves all the character sets of $S_1, \ldots, S_L$ and all single symmetric differences between them.*

Proof. The lemma is derived in a straightforward manner from the construction algorithm. All distinct character sets appearing at $S_i \in S$ are represented by the vertices of V. Regarding the edges, the algorithm systematically detect every single symmetric difference between two nodes of consecutive layers, adding to E edges not included in it yet. □

Observation 4. *Layer f of the graph G constructed according to $S = \{S_1, \ldots, S_L\}$, consists of at most* $O(Ln)$ *distinct nodes.*

Proof. According to Observation 1, the number of cs_f derived from a single string is bounded by $O(n)$. Considering cs_fs from several graphs, their intersection may be empty or very small, as there are $O(n^f)$ possible cs_fs for $|\Sigma| = O(n)$. Therefore, even after eliminating identical cs_fs appearing on distinct intervals from the same string or from different strings, each S_i may contribute at most $n - f + 1$, resulting in $O(L(n - f + 1))$ nodes in layer f of the final graph G. □

Observation 5. *Considering G constructed according to $S = \{S_1, \ldots, S_L\}$, the number of edges between all nodes representing cs_fs (layer f) and nodes representing $cs_(f + 1)$s (layer $f + 1$) is bounded by* $O(Lnf)$.

Proof. According to Observation 4, the number of $cs_(f + 1)$s in G is bounded by $O(Ln)$. A $cs\_(f + 1)$ is constructed from a cs_f to which a symbol is added. Therefore, every $cs\_(f + 1)$ can have at most $f + 1$ distinct edges emanating from cs_fs, yielding $O(Ln)(f + 1)$ possible edges between layer f and layer $f + 1$. □

Lemma 5. *The construction of G by the* CONSTRUCT_GRAPH(S) *procedure for $S = \{S_1, \ldots, S_L\}$, consumes* $O(Ln^3)$ *time and space.*

Proof. The first step of extracting all character sets of S_i, grouped according to their sizes, is done in $O(n^2)$ using the window procedure L times. Sorting $List^f$, including Ln character sets, (line 3) takes $O(Ln \log Ln)$ time and is repeated for every layer, thus n times.

Uniting identical character sets to a single item in $List^f$, costs as the number of intervals copied, yet as there may be maximum Ln intervals with cs_f, this step is done in time $O(Ln)$ for every layer.

The maximal number of distinct character sets is $O(Ln^2)$, according to Observation 4. Constructing V (lines 5–7) is linear in the number of the character sets found.

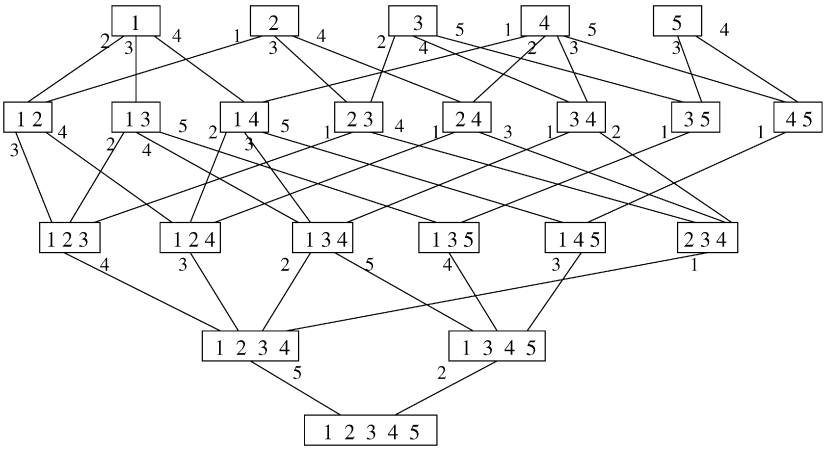

Fig. 4. The Approximate Common Interval graph G.

In order to create edges between layer f and layer $f+1$ we construct a dictionary and preprocess it as in [BG] algorithm (lines 9–10) in time $O(Ln^2)$, linear in the dictionary size (there are $O(Ln)$ $cs_(f+1)$s each of length n). Each of the $O(Ln)$ cs_fs serves as a query text, and performing a query requires $O(n+occ)$ time. Thus, the time required to construct edges between two consecutive layers is $O(Ln^2 + edges)$ and according to Observation 5 it is bounded by $O(Ln^2 + Lnf) = O(Ln^2)$. All in all, having n layers in G, the procedure of constructing both V and E, including edges between all consecutive layers consumes $O(Ln^2) + O(Ln^2 \log n) + O(Ln^3) = O(Ln^3)$ time.

Regarding space: According to Observations 4, $|V| = O(Ln^2)$. Since the nodes contain the character sets they represent, each nodes consumes $O(n)$ space. We also save all intervals bearing distinct character sets, in their corresponding nodes, yet as there are at most Ln^2 such intervals, their preservation, adds no more to the vertices space requirements.

While detecting the edges between layers f, $f+1$ a dictionary and tries consuming $O(Ln^2)$ space are constructed. This procedure is repeated $n-1$ times, yet before constructing the dictionary of the next layer, the previous one can be deleted, thus, merely $O(n^2)$ space is required for the preprocesses done, according to the [BG] algorithm.

Due to Observation 5, the number of edges between consecutive layers cannot exceed $O(Ln^2)$, therefore the total number of edges is bounded by $O(Ln^3)$. The label attached to every edge bears a single character, thus not requiring additional space. All in all we get that the space required for $|V|$ equals the demands of $|E| = O(n^3)$, so the space requirements for the construction and preservation of G is $O(n^3)$. □

We can now continue to answer the ACI problem. First we have the following lemma, equivalent to Lemma 3:

Lemma 6. *Consider graph $G = (V, E)$ constructed according to $S = \{S_1, \ldots, S_L\}$. Given a node $v \in V$ representing cs_v, a node $u \in V$ located at the end of a path of length K, starting in v, where all labels on the path are distinct, stands for a character set cs_u where $cs_v \Delta cs_u = K$.*

Proof. Due to Lemma 4, G constructed according to L strings preserves the single symmetric difference relation, just as the graph constructed upon a single string. Therefore, the proof of Lemma 3 holds for G as well. □

Lemma 7. *The solution to the approximate common Intervals with multiple strings can be obtained by the* ACI(S) *algorithm (Fig. 3), where $S = \{S_1, S_2, \ldots, S_L\}$.*

Proof. Here again, according to Lemma 4, there is no difference between a graph constructed according to a single string and one according to L strings. Therefore, the modified DFS suggested at ACI(S) procedure, would solve the ACI problem for L strings as well. □

Theorem 1. *The Approximate Common Interval problem for L strings can be solved in $O(Ln^3 + occ)$ time and $O(Ln^3)$ space, where n is the length of every string and occ is the size of the output.*

Proof. The construction of G consumes $O(Ln^3)$ time and space as was proved in Lemma 5. Due to Lemma 7 we have that the approximate common intervals of a

certain character set cs, can be obtained by starting a slightly modified DFS traversal from the node representing cs. Since every node encountered is part of the output, including also all intervals associated to that character set, this DFS procedure is linear in the output size. □

4.1. Other types of queries

The suggested graph G is highly informative and can support different types of queries as well. For example, consider the ACI(m) problem:

Input: A set of strings $S = \{S_1, \ldots, S_L\}$ of size n each over alphabet Σ, and constants K, m.

Output: For every distinct character set cs appearing in S, report all character sets cs', occurring in at least m distinct strings in S, such that $cs \Delta cs' \leqslant K$.

In order to answer the ACI(m) problem, we need so slightly modify the suggested ACI algorithm by adding a preprocessing stage: For every node in V, we check whether the intervals associated to it come from m distinct strings and if so we mark the node. This confirmation can be done by simply going over the intervals as they are sorted according to the source string. Since there are at most Ln^2 possible intervals obtained from the input strings, going over them all in order to mark nodes, requires $O(Ln^2)$ time. Having the appropriate nodes marked, we apply the ACI algorithm but report in line 7, merely marked vertices as part of the output.

Answering the original ACI problem costs $O(Ln^3 + occ)$ as was proved in Theorem 1. However, in the ACI(m) problem, since not every node encountered in the DFS traversals is part of the output, the traversals cannot be accounted for the output size, hence the time required for solving the ACI(m) problem in addition to constructing G is $O(L^2n^4)$ in the worst case, for traversing the graph considering each node as a starting point. However, having the gene clusters motivation in mind, usually K is rather small and the time expected would be smaller.

This time will be required for other closely related problems, such as when the demand is to report only character sets with exactly K symmetric difference, or when there is a need to report merely character sets originating in certain S_is. Our graph, whose construction is straightforward, will support these problems and others.

5. Conclusions

In this paper we have dealt with the problem of Approximate Common Interval for multiple strings. We showed a simple procedure for the construction of a graph G representing a single symmetric difference relation between all character sets occurring in the input set of strings. From this graph all character sets (including all the intervals containing them) within K symmetric difference can be attained by a simple traversal over the graph requiring time asymptotic to the size of the output. Our main contribution is providing a simple and versatile algorithm, supporting the approximate common interval problem. Moreover, we solve the problem for general K and improves the space requirements for small Ls.

References

[1] A. Amir, A. Apostolico, G.M. Landau, G. Satta, Efficient text fingerprinting via Parikh mapping, J. Discrete Algebra 26 (2003) 1–13.

[2] M.P. Beal, A. Bergeron, S. Corteel, M. Raffinot, An algorithmic view of gene teams, Theoret. Comput. Sci. 320 (2–3) (2004) 395–418.

[3] G.S. Brodal, L. Gasieniec, Approximate dictionary queries, in: Proc. 7th Annual Symposium on Combinatorial Pattern Matching, in: Lecture Notes in Comput. Sci., vol. 1075, Springer, 1996, pp. 65–74.

[4] X. He, M.H. Goldwasser, Identifying conserved gene clusters in the presence of homology families, J. Comput. Biol. 12 (6) (2005) 638–656.

[5] R. Kolpakov, M. Raffinot, New algorithms for text fingerprinting, in: Proc. 17th Annual Symposium on Combinatorial Pattern Matching, 2006, pp. 342–353.

[6] I.B. Rogozin, K.S. Makarova, J. Muravi, E. Czabarka, Y.I. Wolf, R.L. Tatusov, L.A. Szekely, E.V. Koonin, Connected gene neighborhoods in prokaryotic genomes, Nucl. Acids Res. 30 (2002) 2212–2223.

[7] T. Schmidt, J. Stoye, Quadratic time algorithms for finding common intervals in two and more sequences, in: Proc. 15th Annual Symposium on Combinatorial Pattern Matching, in: Lecture Notes in Comput. Sci., vol. 3109, Springer, 2004, pp. 347–358.

[8] I. Yanai, C. DeLisi, The society of genes: Networks of functional links between genes from comparative genomics, Genome Biol. 3 (64) (2002) 1–12.